07 — Final project: Bookshop

The closing project. It uses everything from modules 01 to 05: forms, validation, sessions, OOP with namespaces and an autoloader, PDO, security. Treat it as a test: build it yourself first, then compare with the solution.

It's also good preparation for the certification, whose exam is made of practical cases like this one.

Allow 3 to 5 hours. Build it in the order below: each part works on its own.

Part 1 — Catalog

Read first: *Bonus: pagination, sorting and filtering* in module 05.

- A `books` table (id, title, author, year, price, created_at), created with sample books on first run (20+, to have several pages).
- Books displayed in a table, **8 per page**, with *Previous / 1 2 3 / Next* links.
- A **search** field filtering on title or author.
- **Sortable columns**: clicking *Title*, *Author*, *Year* or *Price* sorts by that column; clicking again reverses the order. Show an arrow ▲▼ on the current column.
- Search, sort and page work **together**: changing page keeps the search and the sort; a new search or sort goes back to page 1.
- Robustness: `?page=abc`, `?page=999`, `?sort=hack` must not break anything.

Part 2 — Admin

Read first: *Bonus: securing an admin area* in module 05.

- `admin/login.php`: password form (hash + `password_verify`, `session_regenerate_id` after login).
- Every admin page redirects to the login when not logged in.
- `admin/index.php`: the same list, with *Edit* and *Delete* buttons, and an *Add a book* link.
- `admin/edit.php`: one form for both adding (no `id`) and editing (`?id=12`), with validation: title and author required, year optional but valid, price ≥ 0 and accepting `12,50`.
- Delete and logout are **POST** forms, never links.
- All POST forms protected by a **CSRF token**.
- Flash messages after each action (""Dune" updated.").

Part 3 — Visit statistics

Read first: the section below.

- A second table, `visits`, with one row per page view of the **public** catalog.
- For each view, store: date and time, anonymized IP, HTTP method, path, the search term if any, HTTP status code, page generation time in ms, referer, user agent, browser and OS (detected from the user agent), preferred language, and a visitor id.
- Recording the visit must **never break the page**, even if the database fails.
- `admin/stats.php` (protected) showing, for the last 7 days: total page views, distinct visitors, average page time, views per day, top browsers, top systems, top searches, and the 20 latest visits.

Test it by browsing the catalog from two different browsers (or a private window), with and without searches.

Bonus

- Move the app to MySQL (see module 06): only `Database.php` should change, plus a `schema.sql`.
- Add a *Visits* chart: views per day as a bar of `█` characters, or an inline SVG.
- Statistics per book: record which books appear in search results, and show the most searched authors.
- Purge visits older than 30 days automatically (a `DELETE` run on the stats page).

New concepts: knowing who visits

`$_SERVER`: what PHP knows about the request

PHP fills `$_SERVER` with information about the current request. Try `var_dump($_SERVER);` on any page. The useful keys:

Key	Example	Notes
`REMOTE_ADDR`	`203.0.113.42`	Visitor's IP. Behind a proxy or load balancer, it's the proxy's IP
`REQUEST_METHOD`	`GET`	
`REQUEST_URI`	`/index.php?q=dune&page=2`	Path + query string
`HTTP_USER_AGENT`	`Mozilla/5.0 (Windows NT 10.0; ...) Firefox/131.0`	Browser and OS, as **announced** by the browser
`HTTP_REFERER`	`https://www.google.com/`	Previous page. Often missing
`HTTP_ACCEPT_LANGUAGE`	`fr-FR,fr;q=0.9,en;q=0.8`	Preferred languages, in order
`REQUEST_TIME_FLOAT`	`1760685418.5123`	Start of the request, with microseconds
`HTTPS`	`on`	Set when the page is served over HTTPS

Every `HTTP_*` key is a **request header sent by the browser**: the visitor controls it and can put anything in it. Treat it like form input: store it with prepared statements, escape it when displaying it, and never use it for security decisions.

Behind a proxy (Cloudflare, a load balancer), the real IP is usually in the `HTTP_X_FORWARDED_FOR` header. But anyone can send that header, so only read it when you know a trusted proxy is in front of your server.

Splitting a URL

```php
$uri = $_SERVER['REQUEST_URI'];                // /index.php?q=dune&page=2
$path = parse_url($uri, PHP_URL_PATH);         // /index.php
parse_str(parse_url($uri, PHP_URL_QUERY) ?? '', $params);   // ['q' => 'dune', 'page' => '2']
```

Running code after the page: `register_shutdown_function`

To record the HTTP status code (200, 302, 404…) and the total generation time, the recording must happen **at the end** of the page, including when the page stops early with `exit` (a redirect, for example):

```php
$start = microtime(true);

register_shutdown_function(function () use ($start) {
    $status = http_response_code();                          // final status code
    $ms = (int) round((microtime(true) - $start) * 1000);   // duration
    // ... INSERT INTO visits ...
});
```

Statistics must never break the site

Wrap the recording in a `try / catch` and log errors instead of displaying them:

```php
try {
    // INSERT ...
} catch (Throwable $e) {
    error_log('Tracker: ' . $e->getMessage());
}
```

Personal data (GDPR)

In the EU, an IP address is **personal data**. For statistics you rarely need it whole, so anonymize it before storing: `203.0.113.42` → `203.0.113.0`. The same goes for anything that identifies a person. Store what you need, no more. Real sites also need to inform users and, depending on the data, ask for consent.

To count visitors without storing anything personal, give each visitor a random id kept in the session:

```php
$_SESSION['visitor_id'] ??= bin2hex(random_bytes(8));
```

Statistics are just `GROUP BY`

Once the data is in a table, every statistic is one SQL query:

```sql
-- Views and visitors per day
SELECT substr(visited_at, 1, 10) AS day,          -- MySQL: DATE(visited_at)
       COUNT(*) AS views,
       COUNT(DISTINCT visitor_id) AS visitors
FROM visits
GROUP BY day
ORDER BY day DESC;

-- Top 5 browsers
SELECT browser, COUNT(*) AS total
FROM visits
GROUP BY browser
ORDER BY total DESC
LIMIT 5;
```

Detecting the browser from the user agent is messy: Edge's user agent contains "Chrome", Chrome's contains "Safari". The solution tests them in the right order with `match (true)`. Real projects use a library for this.